AICTE INTELLIGENT APPLICATION VERIFICATION PLATFORM

Version 1 — MVP Product, Workflow, Architecture & Technology Document

Submission and implementation reference

Document purpose	Explain the problem, proposed solution, users, workflow, architecture and Version 1 technology stack.
Product approach	Evidence-led verification with an authorized human officer making the official decision.
Core differentiator	DocuPass — reusable verification evidence for eligible documents, subject to validity and current checks.
Version boundary	This document describes Version 1 MVP only.

1. Executive Summary

The AICTE Intelligent Application Verification Platform is a proposed web-based system that helps institutions submit approval-related applications and supporting documents, while helping authorized officers review the submitted information through a structured, evidence-linked workflow.

The platform brings application submission, document storage, text extraction, field comparison, rule-based checks, verification findings, officer review and audit history into one connected system. It is designed to make inconsistencies easier to identify and explain, preserve document versions, and provide a traceable record of checks and decisions.

Its distinguishing feature is DocuPass: a reusable document-verification record for eligible documents. The platform can compare a newly submitted file with a previously verified file and consider reuse only when the document type is eligible, the prior verification record is trustworthy and applicable, and the document remains valid with required current-status checks completed.

The MVP is decision-support software. It does not replace an authorized officer or independently approve/reject applications. It must distinguish a detected mismatch from proven fraud and must show external checks as unresolved when an authorized source is unavailable.

2. Problem Statement

Approval-related applications may include forms, certificates, declarations and other supporting documents. Reviewing these materials can require officers to inspect many files, compare repeated information across documents, check requirements and keep track of corrections or replacement uploads.

The main problems the platform is designed to address are:

- Manual review effort: officers may need to open and inspect many documents separately.
- Information inconsistencies: institution names, addresses, dates, registration details or other fields may differ between the application form and supporting documents.
- Scanned-document difficulty: some documents contain images rather than machine-readable text, making automatic processing harder.
- Duplicate or changed files: the same file may be submitted more than once, while a replacement file may need a new verification run.
- Repeated verification work: eligible documents may need to be checked again in later applications even when a trustworthy prior verification record exists.
- Limited traceability: reviewers need a clear record of what was checked, what evidence supported a finding, who acted and when.
- External verification uncertainty: a department portal or API may not be available, accessible or authorized for automated use.

The platform aims to organize and assist this work. It does not assume that every inconsistency is fraud, and it does not claim authenticity verification unless a real, approved verification method has produced a result.

3. Proposed Solution

Build a shared web platform with two role-based portals: an institution portal and an officer portal. Both use the same Django backend and shared PostgreSQL database. Uploaded originals are held in private document storage, while the backend coordinates processing and records the results.

3.1 Institution portal

- Sign in and access only the institution's permitted records.
- Create an application and select a supported category.
- Complete the category-specific form and upload the required documents.
- Submit the application and track its status.
- View officer comments or clarification requests.
- Upload replacement documents or additional evidence while preserving earlier versions.

3.2 Officer portal

- Sign in with officer permissions.
- View the application queue and application statuses.
- Open an application to inspect form data, original files and document versions.
- Run or review verification checks and inspect evidence-linked findings.
- Request clarification or replacement documents when needed.
- Record authorized review outcomes and reasons.
- View the audit history of relevant actions and verification runs.

3.3 Core verification capabilities

- Extract text from digital PDFs and use OCR for scanned documents.
- Normalize and compare relevant fields between the application form and supporting documents.
- Run category-specific deterministic checks using versioned rules based on the applicable official requirements.
- Detect exact file matches using SHA-256 and flag potential duplicates using text normalization or similarity methods where implemented.
- Record the source, outcome, date, document version and evidence for each check when available.
- Present findings to officers in a report that separates passed checks, detected issues, pending checks, unavailable sources and checks requiring manual review.

4. DocuPass — Reusable Document Verification Passport

DocuPass is the proposed differentiating feature of the platform. Its goal is to avoid repeating every verification step for the same eligible document when reliable prior verification evidence can still be reused.

The guiding principle is: “Verify once, reuse when eligible, and re-verify whenever necessary.”

4.1 How DocuPass works

1. The system processes the document and runs the relevant authenticity and consistency checks available for that document type.
2. After a verification attempt, the platform records the outcome, evidence, source, timestamp, document type, fingerprint, validity details and applicable rule context.
3. The platform signs its verification record using a protected platform key. This signature helps show that the platform record has not been altered; it does not prove that the original document is authentic.
4. When the institution submits a document later, the system calculates its SHA-256 fingerprint and compares it with stored versions.
5. If the file matches, the system evaluates reuse eligibility, validity and whether the earlier verification evidence is still applicable. It performs any required current-status checks.
6. If all required conditions pass, the platform can reuse the prior verification evidence and record a reuse event.

7. If the file differs, the document has expired, a required status check fails, evidence is insufficient or the source is unavailable, the system performs fresh checks, requests updated evidence or routes the case for manual review.

4.2 Essential cryptographic distinction

Mechanism	What it tells us	What it does not tell us
SHA-256 file fingerprint	Whether the file bytes exactly match a previously fingerprinted file.	Whether the document is genuine, valid, unrevoked or suitable for a new application.
Issuer digital signature	Can help verify signer identity and document integrity when the signature and trust chain are properly validated.	It does not by itself prove every fact stated in the document or guarantee current validity.
Platform signature on DocuPass record	Helps establish that a verification record was created by the platform and has not been altered.	It does not prove the original document's authenticity.

4.3 Reuse eligibility conditions

8. The document type is approved for reuse and the earlier verification has an adequate recorded outcome.
9. The document remains within its validity period and required revocation or current-status checks pass.
10. The prior verification record is trustworthy, intact and applicable to the requirements of the new application.

A changed fingerprint means the file is different; it is not proof of fraud. A matching fingerprint means the file is identical; it is not proof that the document remains valid. Reuse rules must follow the current applicable official requirements for each document type.

5. End-to-End System Workflow

Step 1 — Sign in: The institution or authorized officer signs in. Role-based permissions control which actions and records are accessible.

Step 2 — Create an application: The institution chooses a supported category, such as Extension of Approval, increase in student intake or introduction of a new programme, and completes the form.

Step 3 — Upload documents: The institution uploads supporting files. The system validates file type and size, stores the originals privately and records metadata and versions.

Step 4 — Extract information: PyMuPDF extracts text from machine-readable PDFs. OCR is used for scanned pages when needed. Extracted data is associated with the exact file version.

Step 5 — Compare information: The verification engine compares relevant application fields with extracted document fields and with other relevant documents.

Step 6 — Run checks: The engine performs implemented deterministic and category-specific checks, duplicate checks and any real authorized authenticity checks available for the document.

Step 7 — Evaluate DocuPass: The system compares fingerprints and checks whether prior evidence can be reused. It does not skip required validity or current-status checks.

Step 8 — Produce findings: The system stores a verification run and generates a report with outcomes, reasons, evidence references, unresolved checks and manual-review needs.

Step 9 — Officer reviews: The officer examines the report and original documents. The officer can request clarification or replacement documents and record an authorized outcome.

Step 10 — Institution responds: If clarification is requested, the institution uploads additional or replacement evidence. The earlier versions and previous verification runs remain available.

Step 11 — Audit and tracking: The system records important actions, timestamps, actors, reasons and verification references so the application history can be reviewed.

6. System Architecture

The MVP uses a modular-monolith architecture: one Django backend organized into separate Django apps. The institution and officer portals are separate user experiences, not separate backends. This keeps the first version easier to develop, test and maintain.

Layer	Technology	Responsibility
Presentation	Next.js + React + TypeScript	Institution and officer dashboards, forms, upload pages, status tracking and reports.
UI components	Tailwind CSS + shadcn/ui	Consistent layouts, forms, tables, dialogs and accessible interface components.
API and application logic	Django + Django REST Framework	Authentication, authorization, application workflow, document metadata, verification orchestration and review actions.
Data access	Django ORM + Django migrations	Read/write database records and apply controlled schema changes.
Relational data	PostgreSQL	Users, institutions, applications, metadata, findings, rule versions and audit records.
Private file storage	MinIO locally; S3-compatible storage design	Original files and versions, kept separate from relational records.
Document intelligence	PyMuPDF + PaddleOCR; Tesseract benchmark; OpenCV when needed	PDF extraction, OCR and optional image preprocessing.
Verification and DocuPass	Python modules + SHA-256 + cryptography	Field comparisons, rules, fingerprints, signed verification records and reuse eligibility.
Background work	Celery + Redis when enabled	Long-running extraction and verification tasks.
AI assistance	LLM provider adapter; RAG/pgvector if included	Explain findings or answer rulebook questions using retrieved approved material; does not decide applications.
Testing and operations	pytest, pytest-django, Vitest, React Testing Library, Playwright, Docker Compose, GitHub Actions	Automated testing, consistent local setup, version control and CI.

7. Technology-by-Technology Explanation

Next.js — The framework used to build the web frontend. **Used in this project:** Institution portal, officer portal, dashboards, forms, application status pages and reports.

React — The UI library used to create reusable interface components. **Used in this project:** Reusable application forms, tables, document cards, status labels and report panels.

TypeScript — JavaScript with static types that help catch common mistakes during development. **Used in this project:** Frontend data structures, form state and API response types.

Tailwind CSS — Utility-based styling for spacing, layouts, colors and responsive design. **Used in this project:** Consistent page layouts and responsive dashboards.

shadcn/ui — Reusable UI components that can be styled and adapted to the product. **Used in this project:** Buttons, dialogs, form controls, tabs, tables and alerts.

Django — The Python web framework that runs the core backend. **Used in this project:** Models, application workflow, authentication integration, permissions and business logic.

Django REST Framework — A toolkit for creating REST APIs with Django. **Used in this project:** Frontend-to-backend communication, request validation, API permissions and structured responses.

PostgreSQL — A relational database for structured records. **Used in this project:** Application records, users, metadata, verification runs, results and audit events.

Django ORM — A way to work with database records through Python objects. **Used in this project:** Create, query and update database records without manually writing most SQL.

Django migrations — Versioned changes to database structure. **Used in this project:** Create and update tables as application models evolve.

PyMuPDF — A library for working with PDFs and extracting their text. **Used in this project:** Read text and page information from text-based PDFs.

PaddleOCR — An OCR engine that reads text from images and scanned pages. **Used in this project:** Extract text from scanned certificates and other image-based documents.

Tesseract — An alternative OCR engine. **Used in this project:** Benchmark OCR quality and speed against PaddleOCR on representative sample documents.

OpenCV — A computer-vision library. **Used in this project:** Optional preprocessing, such as improving image contrast or correcting document images before OCR.

Python verification modules — Ordinary Python code implementing deterministic comparisons and checks. **Used in this project:** Normalize fields, compare values, run configured rules and produce structured findings.

SHA-256 / hashlib — A cryptographic hash function that creates a fixed-size fingerprint of file bytes. **Used in this project:** Exact file matching for DocuPass and exact duplicate detection.

cryptography — A Python library supporting cryptographic operations. **Used in this project:** Sign platform verification records and validate signatures using appropriate trusted-key and certificate handling.

Celery — A task queue for work that should run outside a web request. **Used in this project:** Run longer extraction or verification jobs asynchronously when needed.

Redis — An in-memory data service commonly used as a Celery message broker. **Used in this project:** Queue and coordinate background jobs in the configured deployment.

MinIO / private S3-compatible storage — Object storage for files rather than structured database rows. **Used in this project:** Store original documents and replacement versions privately.

LLM provider adapter — A backend integration layer that lets the application call a selected language model. **Used in this project:** Generate clear explanations of already-established findings; responses must be grounded in stored results.

RAG + pgvector — Retrieval-augmented generation with vector search in PostgreSQL. **Used in this project:** Optional search across approved AICTE rulebooks and instructions to provide source-grounded answers.

pytest + pytest-django — Python testing tools. **Used in this project:** Test verification rules, API behavior, permissions, DocuPass eligibility and backend workflows.

Vitest + React Testing Library — Tools for testing frontend code and user-interface behavior. **Used in this project:** Test components, forms, status displays and error handling.

Playwright — A browser automation tool for end-to-end testing. **Used in this project:** Test complete journeys such as login, application submission, upload and officer review.

Docker Compose — A tool for running multiple project components together. **Used in this project:** Start the frontend, backend, database, and local storage consistently.

Git + GitHub Actions — Version control and automated CI workflows. **Used in this project:** Collaborate on code, review changes and run tests automatically.

8. Django Backend Module Structure

Django app	What it contains
accounts	Users, roles, login-related behavior and permissions.
institutions	Institution profiles and links to authorized users.
applications	Application categories, form data, status and submission workflow.
documents	Upload metadata, private-storage references, document versions and access rules.
extraction	PDF extraction, OCR jobs and normalized extracted fields.
verification	Verification runs, check results, comparisons and report data.
docupass	File fingerprints, verification passports, signatures, expiry and reuse history.
reviews	Officer review actions, clarification requests and authorized outcomes.
audit	Timestamped records of important user and system actions.
regulatory_rules	Versioned category-specific checks and their references.
notifications	Application status and clarification notifications, if included in MVP.
ai_assistance	Optional model integration and source-grounded rulebook help.

The apps will share one Django project and one PostgreSQL database. Modules should have clear responsibilities so the code remains manageable as features are added.

9. MVP Data and Records

The initial data model is expected to include the following main records:

- User and role: account identity and permitted actions.
- Institution: institution profile and authorized users.
- Application: category, form data, current status, institution and timestamps.
- Document version: application link, document type, storage reference, original filename, content type, size, fingerprint and upload time.
- Extraction result: text/fields extracted from a specific document version and the extraction method/status.
- Verification run: when checks ran, which versions and rules were used, and the overall processing status.
- Verification finding: individual check result, reason, evidence/page/field reference and whether manual review is needed.
- DocuPass record: fingerprint, document category, prior verification result, evidence/source, timestamps, validity data, signed record and reuse history.
- Review action: officer action, reason, actor and timestamp.
- Audit event: actor or system process, action, related record, timestamp and relevant result.

10. Verification Result Statuses

Status	Meaning
Passed	The specific check completed and its defined condition passed.
Issue detected	The check found an inconsistency or a condition that needs attention.

Pending	The check has not completed yet.
Source unavailable	A required source or service could not be reached; this is not proof of fraud.
Manual review required	A person must examine the evidence or resolve an ambiguity.
Not applicable	The check does not apply to this document or application.
Expired	The relevant document is outside its recorded validity period.
Revoked	A reliable source indicates that the document or credential has been revoked, where such a check is supported.

The report should not collapse every outcome into a single pass/fail label. A source outage, an OCR failure, a mismatch and a confirmed validity problem mean different things and should remain distinguishable.

11. Security, Privacy and Responsible Decision-Making

- Enforce server-side role-based permissions on every protected API and file access.
- Validate actual file content, permitted file types, size limits and processing constraints; do not trust a filename or browser-supplied MIME type alone.
- Keep documents in private storage and issue access only to authorized users. Use short-lived links only when appropriate.
- Use secure sessions/cookies, CSRF protection, HTTPS in deployment, appropriate password hashing and environment-managed secrets.
- Preserve document versions and connect each verification result to the exact file version checked.
- Limit access to sensitive extracted text and avoid sending documents to external AI services unless approved, necessary and disclosed in the system's data-handling design.
- Protect signing keys and define a key-rotation and recovery approach before production use.
- Record audit events and restrict who can change or access audit data. Stronger tamper-evident storage may be required for production.
- Use the applicable official rules and document requirements. Rule sources and versions should be recorded so an officer can understand which requirement was applied.
- Keep the final application decision with the authorized officer. AI-generated text is assistance, not a legal or regulatory decision.

12. MVP Scope and Boundaries

Included in Version 1 MVP

- Institution and officer interfaces connected to one backend.
- Role-based authentication and application access controls.
- Application creation, submission, document upload and status tracking.
- Private document storage and document version history.
- PDF text extraction and OCR for scanned documents.
- Initial cross-document comparisons, exact duplicate checks and configured rule checks.
- Structured verification findings with reasons and evidence references where available.
- DocuPass fingerprint matching, verification-record integrity and reuse eligibility logic for supported document types.
- Officer review actions, clarification workflow and audit events.
- Automated tests for core workflows and security-sensitive behavior.

Dependencies and limitations

- Real authenticity verification depends on available issuer signatures, authorized department portals/APIs or another approved verification process.
- If an external source is unavailable or no integration permission exists, the system must report that check as unresolved or requiring manual review.
- OCR can make mistakes; important extracted values should be reviewable and tied to source pages or regions where feasible.
- Reuse eligibility must be configured according to applicable official requirements; not every document should be reusable.
- The MVP is not a replacement for official review and does not autonomously approve or reject applications.

13. Suggested MVP Acceptance Checklist

- ☐ An institution can sign in and create an application.
- ☐ An institution can upload required documents and later view their status.
- ☐ An institution cannot access another institution's private application or files.
- ☐ An authorized officer can open an application and inspect the original documents.
- ☐ The system can extract text from a digital PDF and OCR a representative scanned sample.
- ☐ The system can compare configured fields and show a clear reason when values differ.
- ☐ Each verification run identifies the document versions and rules used.
- ☐ The report distinguishes issues, pending checks, unavailable sources and manual-review requirements.
- ☐ DocuPass recognizes an exact file match but does not treat that match alone as proof of authenticity or validity.
- ☐ Expired, changed or otherwise ineligible documents trigger the configured fresh-check or manual-review path.
- ☐ Replacement uploads preserve earlier versions and their verification history.
- ☐ Important user and officer actions create audit events.
- ☐ Tests cover permissions, upload validation, verification logic and the main application workflow.

14. Final Technology Stack — Version 1 MVP

Area	Selected technology
Frontend	Next.js + React + TypeScript
UI styling/components	Tailwind CSS + shadcn/ui
Backend	Django + Django REST Framework
Database	PostgreSQL + Django ORM + Django migrations
Document storage	MinIO locally; private S3-compatible object storage design
PDF extraction	PyMuPDF
OCR	PaddleOCR, benchmarked against Tesseract
Image preprocessing	OpenCV only when needed
Verification	Python comparison modules + versioned regulatory rules
DocuPass	SHA-256 via hashlib + cryptography for signed platform records
Background tasks	Celery + Redis when required for long-running processing
Optional AI support	LLM provider adapter; RAG + pgvector only if included in MVP scope
Testing	pytest + pytest-django + Vitest + React Testing Library + Playwright
API documentation	drf-spectacular / OpenAPI
Development and CI	Docker Compose + Git + GitHub Actions

15. Conclusion

The Version 1 MVP will provide a single, traceable workflow for submitting applications, processing documents, identifying inconsistencies, recording verification evidence and supporting authorized officer review. DocuPass adds a structured approach to reusing verification evidence for eligible documents without confusing file matching with authenticity or current validity.

The implementation should prioritize a reliable end-to-end workflow, correct access controls, explainable findings, preserved document versions and honest reporting of unresolved checks. Advanced automation should be added only where the required evidence, rules and integrations can support it.